

PART A • SETTING THE SCENE

1 Overview

This document turns the Verilog 8 lecture notes (26 slides, a mix of slides, whiteboard sketches and waveform screenshots) into one structured reference. It follows the same story as the lecture: **instruction set** → **instruction cycle** → **pipelined datapath** → **Verilog model** → **test programs**, and then extends it with a **pipelined RISC-V (RV32I) core** that fixes the biggest limitation of the lecture design: it needs no dummy instructions, because hazards are handled in hardware.

The five steps every instruction goes through

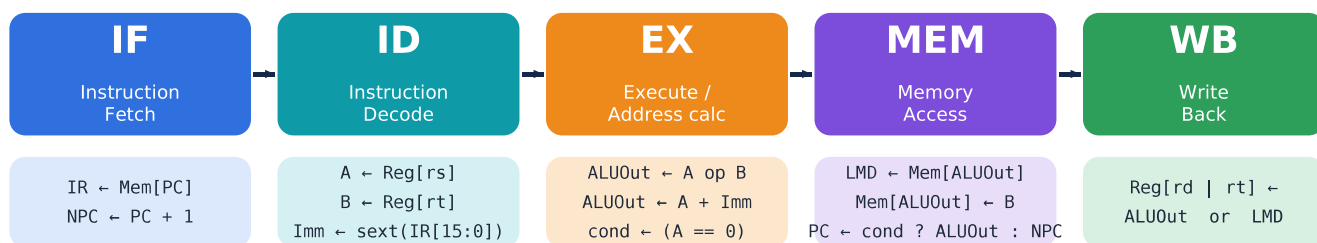


Figure 1. The five-step instruction cycle with its generic micro-operations (MIPS32 notation from the notes).

Roadmap

Chapter	What you get	Source
2 MIPS32 subset	Registers, instruction subset, R / I / J encodings, opcode tables, addressing modes	Slides 1-4
3 Instruction cycle	Micro-operations per stage, worked examples, non-pipelined datapath	Slides 4-9
4 Pipelining	Timing, pipeline registers, per-stage micro-ops, pipelined datapath, hazards	Slides 10-15
5 Verilog model	Complete behavioural model of the 5-stage pipeline, stage by stage	Slides 16-19
6 Test programs	Three programs (sum, memory add, factorial) with machine code and simulation output	Slides 20-26
7 RISC-V core	RV32I formats, subset, pipeline with forwarding / stall / flush, RTL, tests, CPI analysis	New
8 Reference	Side-by-side comparison, cheat sheet, glossary, ideas to go further	New
Appendix	Tiny Python assemblers, how to run everything	New

Everything shown was actually simulated

Both Verilog designs were compiled and run with Icarus Verilog 12. The MIPS32 results reproduce the numbers on the original slides ($R5 = 55$, $\text{Mem}[121] = 130$, $7! = 5040$). The RISC-V core was tested with a loop-and-forwarding program and a Fibonacci program; cycle counts match the analytical hazard model exactly (Section 7.8).

Small corrections to the notes

(1) Example 1 text says the third number is 30, but the program, and the shown output ($R3 = 25$, $R5 = 55$), use 25. (2) The dummy instruction hex 0ce77800 on the slides decodes to OR R15,R7,R7 ($rd = 15$), not R7. It is harmless because R15 is unused; this document uses the exact encoding 0ce73800 for OR R7,R7,R7.

PART A · INSTRUCTION SET ARCHITECTURE

2 The MIPS32 subset

MIPS32 is a classic **RISC** (Reduced Instruction Set Computer) architecture: 32-bit data path, fixed 32-bit instructions, a load/store memory model and very few addressing modes. The lecture implements only a small subset, enough to run real loops and memory programs.

2.1 Registers and memory model

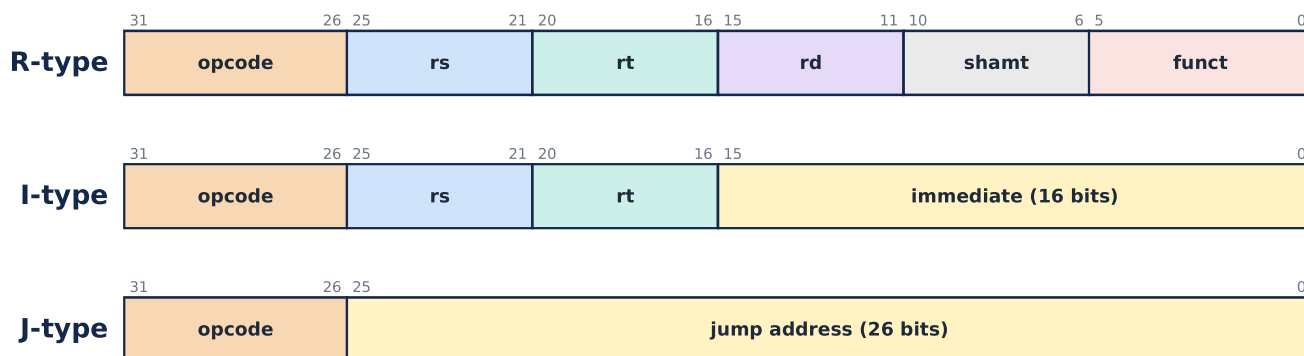
- **32 general-purpose registers** R0 ... R31, each 32 bits. **R0 is hard-wired to 0** and cannot be written.
- A separate 32-bit **program counter (PC)** points to the next instruction. There are **no flag registers** (no zero / carry / sign flags).
- Only **load and store** instructions touch memory; everything else works register-to-register.
- Memory is modelled as **word-addressable** (one 32-bit word per address), so the PC increments by 1. Real byte-addressable MIPS increments by 4.

2.2 Instruction subset

Class	Example	Meaning
Load	LW R2, 124(R8)	$R2 = \text{Mem}[R8 + 124]$
Store	SW R5, -10(R25)	$\text{Mem}[R25 - 10] = R5$
ALU (reg)	ADD R1, R2, R3	$R1 = R2 + R3$
	SUB R12, R10, R8	$R12 = R10 - R8$
	AND R20, R1, R5 / OR R11, R5, R6	bitwise AND / OR
	MUL R5, R6, R7	$R5 = R6 \times R7$
	SLT R5, R11, R12	$R5 = 1$ if $R11 < R12$, else 0
ALU (imm)	ADDI R1, R2, 25	$R1 = R2 + 25$
	SUBI R5, R1, 150	$R5 = R1 - 150$
	SLTI R2, R10, 10	$R2 = 1$ if $R10 < 10$, else 0
Branch	BEQZ R1, Loop	branch to Loop if $R1 = 0$
	BNEQZ R5, Label	branch to Label if $R5 \neq 0$
Jump	J Loop	unconditional (defined, <i>not implemented</i> in the model)
Misc	HLT	halt execution

2.3 Instruction encoding

All instructions are 32 bits wide and fall into three formats. The register fields sit in the **same bit positions** in every format, which is what makes decoding trivially simple and lets the register file be read in parallel with opcode decoding.



shamt / funct unused in this subset; J-type is defined but not implemented in the model.

Figure 2. MIPS32 instruction formats. The opcode occupies bits 31:26 in every format.

R-type (opcode)	Code	I-type (opcode)	Code
ADD	000000	LW	001000
SUB	000001	SW	001001
AND	000010	ADDI	001010
OR	000011	SUBI	001011
SLT	000100	SLTI	001100
MUL	000101	BNEQZ	001101
HLT	111111	BEQZ	001110
		J (J-type)	010000 (not implemented)

Worked encodings

Instruction	Fields	Hex
SUB R5,R12,R25	000001 01100 (rs=12) 11001 (rt=25) 00101 (rd=5) 00000 000000	05992800
LW R20,84(R9)	001000 01001 (rs=9) 10100 (rt=20) 0000000001010100 (84)	21340054
BEQZ R25,Label	001110 11001 (rs=25) 00000 (unused) YYYYYYYYYYYYYY (offset)	3b20YYYY

2.4 Addressing modes

Mode	Example	How the operand / target is found
Register	ADD R1,R2,R3	operands are in registers
Immediate	ADDI R1,R2,200	16-bit constant inside the instruction (sign-extended)
Base + offset	LW R5,150(R7)	R7 + sign-extended offset = memory address
PC-relative	BEQZ R3,Label	16-bit offset added to the (next) PC
Pseudo-direct	J Label	26-bit field forms the jump target

Sign extension

A 16-bit immediate is widened to 32 bits by replicating bit 15 into the upper 16 bits: $\{16\{IR[15]}\} \## IR[15:0]$. In 2's complement this preserves the value: +5 becomes 0x00000005 and -5 (0xFFFB) becomes 0xFFFFFFF5.

PART A • EXECUTING ONE INSTRUCTION

3 The instruction cycle

Before pipelining, consider one instruction at a time. Its execution is divided into five steps, each of which uses a different part of the hardware. Temporary registers (**IR**, **NPC**, **A**, **B**, **Imm**, **ALUOut**, **LMD**, **cond**) carry values from one step to the next.

3.1 Micro-operations per step

Step	Reg-Reg ALU	Reg-Imm ALU	Load	Store	Branch
IF	$IR \leftarrow \text{Mem}[PC]; \text{NPC} \leftarrow PC + 1$				
ID	$A \leftarrow \text{Reg}[rs]; B \leftarrow \text{Reg}[rt]; \text{Imm} \leftarrow \{16\{\text{IR}[15]\}\} \# \text{IR}[15:0]$				
EX	$\text{ALUOut} \leftarrow A \text{ func } B$	$\text{ALUOut} \leftarrow A \text{ func } \text{Imm}$	$\text{ALUOut} \leftarrow A + \text{Imm}$	$\text{ALUOut} \leftarrow A + \text{Imm}$	$\text{ALUOut} \leftarrow \text{NPC} + \text{Imm}$ $\text{cond} \leftarrow (A == 0)$
MEM	$PC \leftarrow \text{NPC}$	$PC \leftarrow \text{NPC}$	$PC \leftarrow \text{NPC}$ $\text{LMD} \leftarrow \text{Mem}[\text{ALUOut}]$	$PC \leftarrow \text{NPC}$ $\text{Mem}[\text{ALUOut}] \leftarrow B$	if (cond) $PC \leftarrow \text{ALUOut}$ else $PC \leftarrow \text{NPC}$
WB	$\text{Reg}[rd] \leftarrow \text{ALUOut}$	$\text{Reg}[rt] \leftarrow \text{ALUOut}$	$\text{Reg}[rt] \leftarrow \text{LMD}$	–	–

Notes: **func** stands for the ALU operation selected by the opcode. For a byte-addressable memory the NPC calculation would be $PC + 4$. Only loads, stores and branches use the MEM step; all other instructions simply pass through it.

3.2 Worked examples

Step	ADD R2,R5,R10	ADDI R2,R5,150	LW R2,200(R6)	SW R3,25(R10)	BEQZ R3,Label
IF	$IR \leftarrow \text{Mem}[PC]$ $\text{NPC} \leftarrow PC+1$	$IR \leftarrow \text{Mem}[PC]$ $\text{NPC} \leftarrow PC+1$	$IR \leftarrow \text{Mem}[PC]$ $\text{NPC} \leftarrow PC+1$	$IR \leftarrow \text{Mem}[PC]$ $\text{NPC} \leftarrow PC+1$	$IR \leftarrow \text{Mem}[PC]$ $\text{NPC} \leftarrow PC+1$
ID	$A \leftarrow \text{Reg}[rs]$ $B \leftarrow \text{Reg}[rt]$	$A \leftarrow \text{Reg}[rs]$ $\text{Imm} \leftarrow \text{sxt}(\text{IR})$	$A \leftarrow \text{Reg}[rs]$ $\text{Imm} \leftarrow \text{sxt}(\text{IR})$	$A \leftarrow \text{Reg}[rs]$ $B \leftarrow \text{Reg}[rt]$ $\text{Imm} \leftarrow \text{sxt}(\text{IR})$	$A \leftarrow \text{Reg}[rs]$ $\text{Imm} \leftarrow \text{sxt}(\text{IR})$
EX	$\text{ALUOut} \leftarrow A + B$	$\text{ALUOut} \leftarrow A + \text{Imm}$	$\text{ALUOut} \leftarrow A + \text{Imm}$	$\text{ALUOut} \leftarrow A + \text{Imm}$	$\text{ALUOut} \leftarrow \text{NPC} + \text{Imm}$ $\text{cond} \leftarrow (A == 0)$
MEM	$PC \leftarrow \text{NPC}$	$PC \leftarrow \text{NPC}$	$PC \leftarrow \text{NPC}$ $\text{LMD} \leftarrow \text{Mem}[\text{ALUOut}]$	$PC \leftarrow \text{NPC}$ $\text{Mem}[\text{ALUOut}] \leftarrow B$	if (cond) $PC \leftarrow \text{ALUOut}$
WB	$\text{Reg}[rd] \leftarrow \text{ALUOut}$	$\text{Reg}[rt] \leftarrow \text{ALUOut}$	$\text{Reg}[rt] \leftarrow \text{LMD}$	–	–

3.3 The non-pipelined datapath

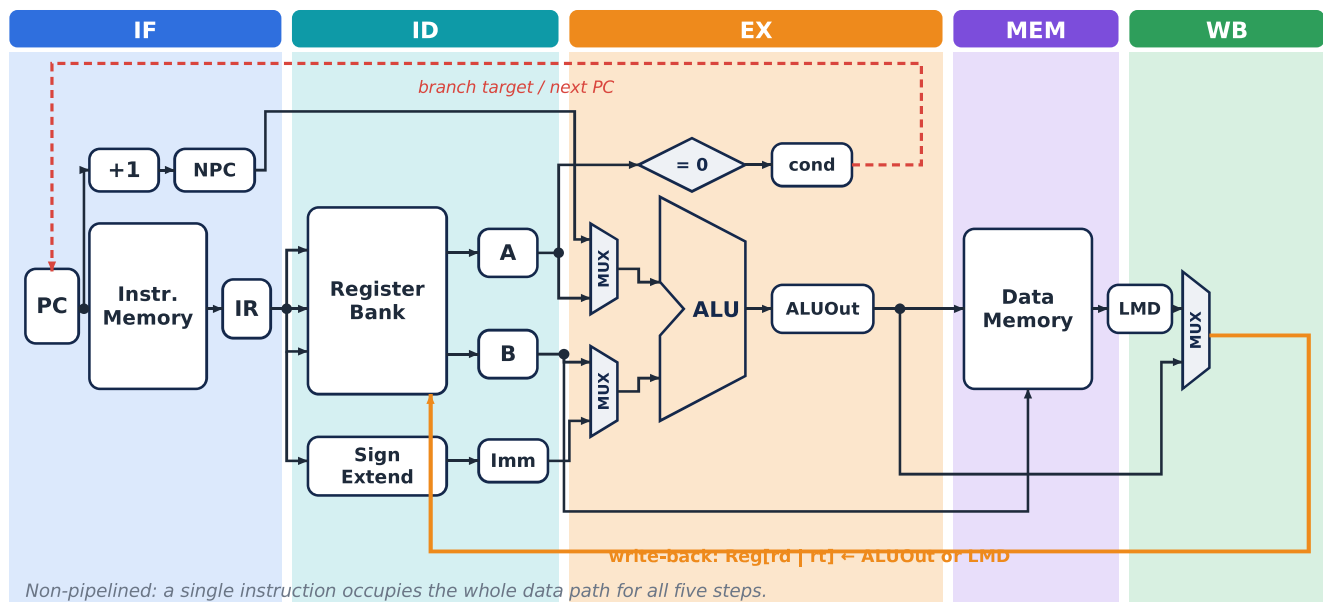


Figure 3. Single-instruction datapath. The orange loop is the write-back path, the red dashed loop updates the PC after a taken branch.

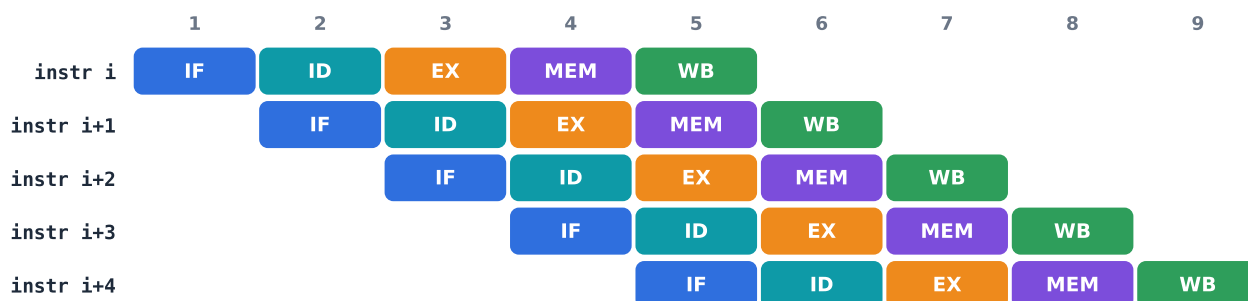
Every stage is idle while the others work: at any moment only one fifth of the hardware is busy. Pipelining fixes exactly that.

PART A • OVERLAPPING INSTRUCTIONS

4 Pipelining the datapath

To pipeline the datapath, every step becomes a **pipeline stage** that must finish within one clock period, and a new instruction is started every period. Registers placed between the stages (the **pipeline latches**) hold each instruction's intermediate values so that five instructions can be in flight at once.

Ideal 5-stage pipeline: one instruction completes every clock cycle



Cycle 5: five instructions are in flight, one per stage → throughput approaches 1 instruction / cycle.

Figure 4. Ideal pipeline timing. From cycle 5 onwards one instruction completes per cycle.

4.1 Pipeline registers and naming convention

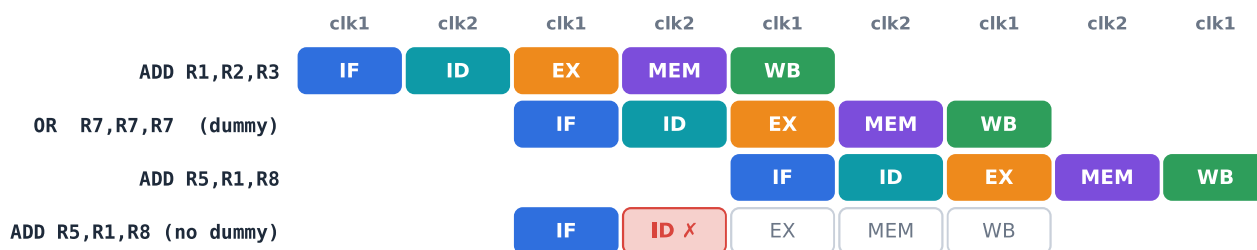
The notes name every temporary register by the latch it lives in: IF_ID, ID_EX, EX_MEM, MEM_WB. For example ID_EX_A is register A, implemented as part of the ID/EX latch.

Latch	Registers it holds
IF_ID	IF_ID_IR, IF_ID_NPC
ID_EX	ID_EX_IR, ID_EX_NPC, ID_EX_A, ID_EX_B, ID_EX_Imm, ID_EX_type
EX_MEM	EX_MEM_IR, EX_MEM_ALUOut, EX_MEM_B, EX_MEM_cond, EX_MEM_type
MEM_WB	MEM_WB_IR, MEM_WB_ALUOut, MEM_WB_LMD, MEM_WB_type

The instruction register (IR) travels with the instruction through all latches. Its fields (rd, rt) are needed again in the WB stage to decide *where* to write the result. The extra type register is a 3-bit class code (R-R ALU, R-I ALU, LOAD, STORE, BRANCH, HALT) produced in ID so later stages do not have to re-decode the opcode.

4.2 Micro-operations of the pipelined stages

Stage	Micro-operations
IF	IF_ID_IR ← Mem[PC] IF_ID_NPC, PC ← (EX_MEM_IR is branch and branch taken) ? EX_MEM_ALUOut + 1 : PC + 1
ID	ID_EX_A ← Reg[IF_ID_IR[rs]] ID_EX_B ← Reg[IF_ID_IR[rt]] ID_EX_NPC ← IF_ID_NPC ID_EX_IR ← IF_ID_IR ID_EX_Imm ← sign-extend(IF_ID_IR[15:0])
EX	R-R ALU: EX_MEM_IR ← ID_EX_IR; EX_MEM_ALUOut ← ID_EX_A func ID_EX_B R-I ALU: EX_MEM_IR ← ID_EX_IR; EX_MEM_ALUOut ← ID_EX_A func ID_EX_Imm LOAD/STORE: EX_MEM_ALUOut ← ID_EX_A + ID_EX_Imm; EX_MEM_B ← ID_EX_B BRANCH: EX_MEM_ALUOut ← ID_EX_NPC + ID_EX_Imm; EX_MEM_cond ← (ID_EX_A == 0)
MEM	ALU: MEM_WB_IR ← EX_MEM_IR; MEM_WB_ALUOut ← EX_MEM_ALUOut LOAD: MEM_WB_LMD ← Mem[EX_MEM_ALUOut] STORE: Mem[EX_MEM_ALUOut] ← EX_MEM_B
WB	R-R ALU: Reg[MEM_WB_IR[rd]] ← MEM_WB_ALUOut R-I ALU: Reg[MEM_WB_IR[rt]] ← MEM_WB_ALUOut LOAD: Reg[MEM_WB_IR[rt]] ← MEM_WB_LMD

RAW hazard avoided with a dummy instruction (stages alternate clk1 / clk2)

R1 is written in WB on clk1 (column 5). With the dummy, the dependent ADD reaches ID on clk2 (column 6) and reads the new value. Without the dummy (bottom row) the dependent ADD would read R1 in column 4, before the write: stale data.

Figure 6. Why one dummy instruction is enough in the lecture design: every stage runs on alternating clock phases.

Why does one dummy suffice? (two-phase clock)

The model uses two non-overlapping clocks. IF, EX and WB are triggered on clk1; ID and MEM on clk2. Consecutive stages therefore sit half a clock period apart. A producer writes the register file in WB on clk1; an instruction two slots behind reads the register file in ID on clk2 of the *following* half-period, i.e. after the write. An instruction directly behind the producer would read too early, so exactly one dummy between a dependent pair is needed.